
系统开发工具基础

实验报告

实验周次：第 2 周

学 院 信息科学与工程学部

专 业 计算机科学与技术

姓 名 臧文商

学 号 25020007160

实验日期 2026.8.31

一、 课堂练习

1. 题目一

(1) 题目内容

第 5 题 控制一个可清理的后台任务

主题：命令行环境：进程、信号与任务控制 建议用时：12—15 分钟

将下方脚本保存为 q05/worker.sh，再完成任务控制。

给定内容

```
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
  echo "$n"
  n=$((n+1))
  sleep 1
done
```

题目要求

- 赋予脚本执行权限，在前台启动，并把 stdout、stderr 分别写入 stdout.log、stderr.log。
- 使用 Ctrl-Z 挂起任务，再用 bg 让它在后台继续；使用 jobs 确认状态。
- 使用 jobs -p 或等价方式取得 PID，不得手工抄写 PID；发送 SIGTERM 并等待任务结束。
- 确认 cleanup.log 中出现 CLEAN_EXIT，且任务已经退出；禁止使用 kill -9。

(2) 解题过程

克隆脚本，使用 chmod +x 赋予脚本执行权限，运行脚本将 stdout、stderr 分别写入 stdout.log、stderr.log，使用 Ctrl+Z 挂起任务，用 bg 让它在后台继续，用 jobs 查看任务状态，jobs -p 获取 PID，kill -s SIGTERM 发送终止信号，cat 查看 cleanup.log 验证清理逻辑。

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ vi worker.sh

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ chmod +x worker.sh

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ ./worker.sh > stdout.log 2> stderr.log

[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ jobs -p
1670

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ SIGTERM
bash: SIGTERM: command not found

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ kill -s SIGTERM $(jobs -p)

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q05 (main)
$ cat cleanup.log
CLEAN_EXIT
[1]+  Done                    ./worker.sh > stdout.log 2> stderr.log
```

(3) 实验结果

完成任务控制，如上图。

2. 题目二

(1) 题目内容

第 6 题 语义重构与本地开发反馈

主题：开发环境与工具 建议用时：12—15 分钟

在 q06 中按照给定代码创建三个 Python 文件，并使用编辑器语言服务器完成一次语义重构。

给定内容

```
# math_utils.py
def total_price(price: float, count: int) -> float:
    return price * count

# app.py
from math_utils import total_price
print(total_price(12.5, 4))

# test_math_utils.py
from math_utils import total_price
def test_total_price():
    assert total_price(12.5, 4) == 50.0
```

题目要求

第 4 页

系统开发工具基础 · 每周课上实验检查题

- 在已配置好 Python、pytest 和 ruff 的课程环境中打开 q06，并启用 Python 语言服务器。
- 分别演示跳转到定义和查找引用。
- 使用“重命名符号”把 total_price 改为 calculate_total，保证定义和所有代码引用同步改变。
- 在 app.py 中临时加入未使用的 import os，用 ruff 发现并删除；最后运行 ruff 和 pytest 并保证通过。

(2) 解题过程

先配置 pytest 和 ruff。

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ python -m pip install pytest ruff
Collecting pytest
  Downloading pytest-8.4.2-py3-none-any.whl (365 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 365.8/365.8 KB 437.8 kB/s eta 0:00:00
Collecting ruff
  Downloading ruff-0.16.5-py3-none-win_amd64.whl (10.5 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.5/10.5 MB 3.4 MB/s eta 0:00:00
Collecting tomli>=1
  Downloading tomli-2.4.1-py3-none-any.whl (14 kB)
Collecting pygments>=2.7.2
  Downloading pygments-2.21.0-py3-none-any.whl (1.3 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.3/1.3 MB 6.6 MB/s eta 0:00:00
```

验证环境并建立初始文件。

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ python -m pytest --version
pytest 8.4.2

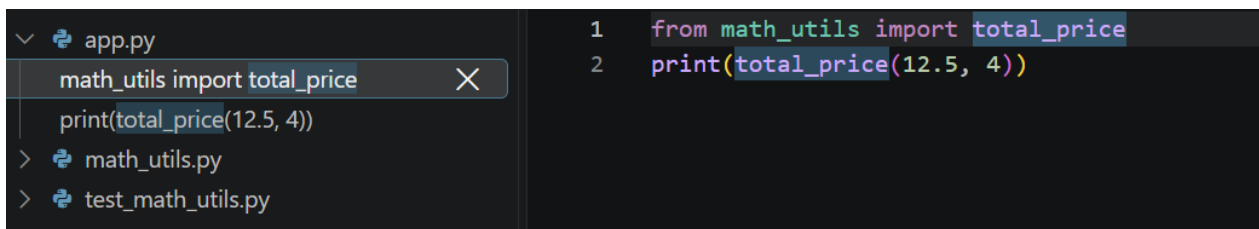
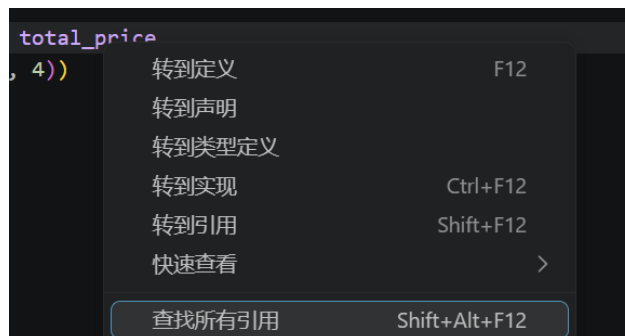
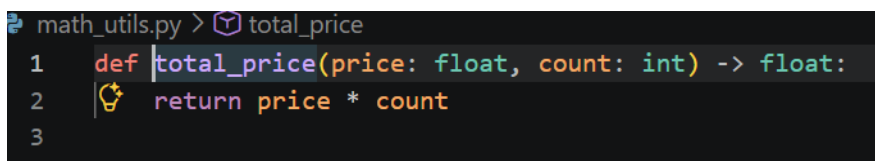
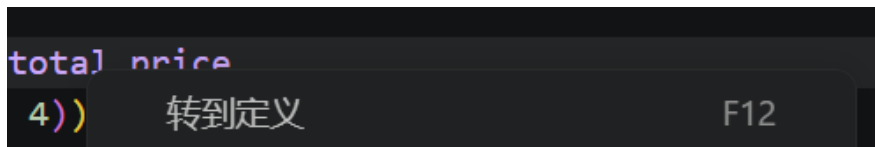
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ python -m ruff --version
ruff 0.16.5

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ vi math_utils.py

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ vi app.py

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ vi test_math_utils.py
```

使用 VS Code 打开项目，启用 Python 语言服务器，分别进行“跳转到定义”“查找引用”和“重命名”操作如下图。



```

math_utils.py > total_price
1 def total_price(price: float, count: int) -> float:
2
3
calculate_total
按 Enter 进行重命名, 按 Ctrl+Enter 进行预览

```

在 app.py 中加入未使用的 import os, 用 ruff 发现。

```

$ python -m ruff check .
1001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
1 | / import os
2 | | from math_utils import calculate_total
3 | | print(calculate_total(12.5, 4))
   | ^
help: Organize imports
1 | import os
2 | +
3 | from math_utils import calculate_total
4 | +
5 | print(calculate_total(12.5, 4))

F401 [*] 'os' imported but unused
--> app.py:1:8
1 | import os
   | ^^
2 | from math_utils import calculate_total
3 | print(calculate_total(12.5, 4))
   |
help: Remove unused import: 'os'
1 | - import os
   |
2 | from math_utils import calculate_total

1001 [*] Import block is un-sorted or un-formatted
--> test_math_utils.py:1:1
1 | from math_utils import calculate_total
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
2 | def test_total_price():
3 |     assert calculate_total(12.5, 4) == 50.0
   |
help: Organize imports
1 | from math_utils import calculate_total
2 | +
3 | +
4 | def test_total_price():
   |

Found 3 errors.
[*] 3 fixable with the '--fix' option.

```

用 ruff 修正后, 运行 ruff 和 pytest。

```

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ python -m ruff check --fix .
Found 3 errors (3 fixed, 0 remaining).

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ python -m ruff check .
All checks passed!

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q06 (main)
$ python -m pytest
===== test session starts =====
platform win32 -- Python 3.9.13, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\guolicheng\Desktop\系统开发工具基础\实验\week2\q06
collected 1 item

test_math_utils.py .

===== 1 passed in 0.01s =====

```

(3) 实验结果

成功完成跳转定义、查找引用、重命名等操作; 加入 import os 后用 ruff 发现并删除, 最后运行 ruff 检查无报错, pytest 单元测试全部通过。

3. 题目三

(1) 题目内容

第 7 题 用调试器定位归并排序缺陷

主题: Debugging 建议用时: 12—15 分钟

将下列存在缺陷的归并排序代码保存为 q07/merge_sort.py, 再使用调试器定位并修复。

给定内容

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1 # 此处存在缺陷
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

题目要求

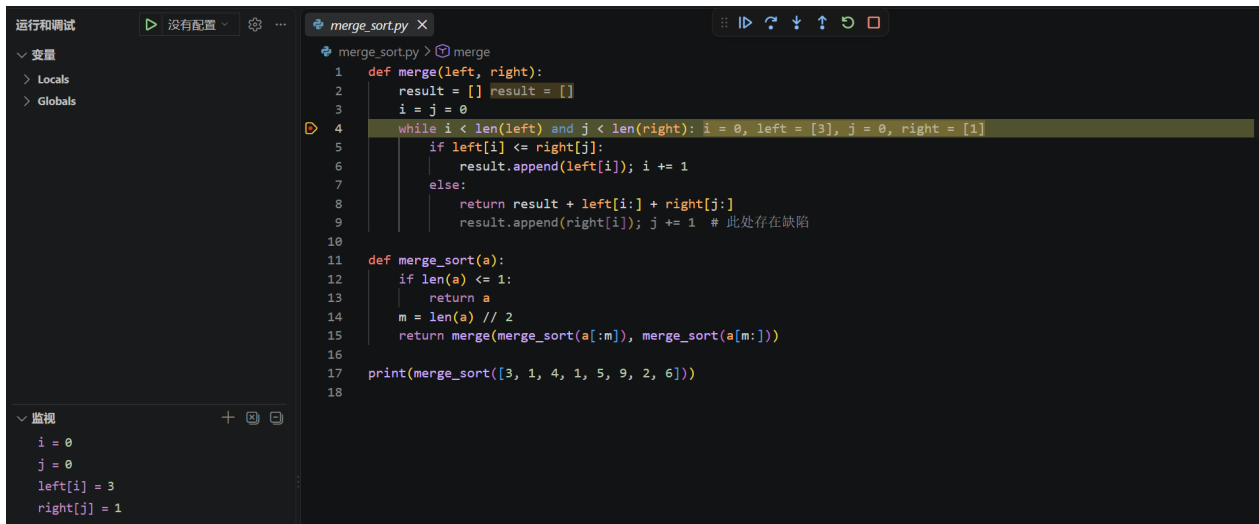
- 先运行给定输入, 确认结果错误或出现异常。
- 使用 pdb、IDE debugger 或等价调试器, 在 merge 中设置断点并观察 i、j、left[i]、right[j]。
- 完成最小修复, 不得用 sorted 替换归并排序。
- 使用 pytest 增加两个测试: 给定输入和含重复元素的列表; 保证测试通过。

(2) 解题过程

先运行给定输入, 确认出现异常。

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q07 (main)
$ python merge_sort.py
Traceback (most recent call last):
  File "C:\Users\guolicheng\Desktop\系统开发工具基础\实验\week2\q07\merge_sort.py", line 17, in <module>
    print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
  File "C:\Users\guolicheng\Desktop\系统开发工具基础\实验\week2\q07\merge_sort.py", line 15, in merge_sort
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))
  File "C:\Users\guolicheng\Desktop\系统开发工具基础\实验\week2\q07\merge_sort.py", line 15, in merge_sort
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))
  File "C:\Users\guolicheng\Desktop\系统开发工具基础\实验\week2\q07\merge_sort.py", line 4, in merge
    while i < len(left) and j < len(right):
TypeError: object of type 'NoneType' has no len()
```

使用 VS Code, 在 merge 中设置断点并观察 i,j,left[i],right[j], 发现异常来源于: else 分支提前 return; 部分分支无返回; right 数组下标错用 i, 应使用 j。



针对上述问题进行最小修复如下图。

```

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1 # 此处存在缺陷
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))

```

使用 pytest 增加两个测试。

```

from merge_sort import merge_sort

# 题目给定输入
def test_given_input():
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]

# 含重复元素的列表
def test_dup_elements():
    assert merge_sort([5, 2, 2, 8, 2, 1]) == [1, 2, 2, 2, 5, 8]

```

(3) 实验结果

测试通过，异常解决。

```

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q07 (main)
$ python merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q07 (main)
$ vi test_merge.py

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q07 (main)
$ python -m pytest test_merge.py -v

===== test session starts =====
platform win32 -- Python 3.9.13, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\guolicheng\AppData\Local\Programs\Python\Python39\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\guolicheng\Desktop\系统开发工具基础\实验\week2\q07
collected 2 items

test_merge.py::test_given_input PASSED [ 50%]
test_merge.py::test_dup_elements PASSED [100%]

===== 2 passed in 0.02s =====

```

4. 题目四

(1) 题目内容

第 8 题 先测量，再优化慢速词频程序

主题: Profiling 建议用时: 12—15 分钟

在 q08 中创建可重复生成的词表和故意低效的去重程序，再使用性能分析工具优化。

给定内容

```
# generate_words.py
import random
```

第 5 页

系统开发工具基础 · 每周课上实验检查题

```

random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))

# wordfreq.py
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))

```

题目要求

- 生成 words.txt，使用 time 运行原程序 2 次并记录耗时。
- 使用 cProfile -s cumulative 运行一次，找出主要耗时位置。
- 在不改变最终 count 的前提下优化程序；不得写死 1000。
- 使用相同输入再运行 2 次，计算优化前后中位数和加速比。

(2) 解题过程

建立初始文档后，运行 generate_words.py 生成 words.txt，使用 time 运行原程序两次，记录耗时分别为 1.635s 和 1.639s。


```

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ python generate_words.py

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m1.635s
user    0m0.000s
sys     0m0.062s

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m1.629s
user    0m0.000s
sys     0m0.046s

```

使用 cProfile-s cumulative 运行一次，找出主要耗时位置为列表 append 操作与列表成员判断。

```

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ python -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.101 seconds

ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000    0.000    0.101    0.101 {built-in method builtins.exec}
1      0.099    0.099    0.101    0.101 wordfreq.py:1(<module>)
1      0.001    0.001    0.001    0.001 {method 'split' of 'str' objects}
1      0.000    0.000    0.000    0.000 {method 'read' of '_io.TextIOWrapper' objects}
1      0.000    0.000    0.000    0.000 {built-in method io.open}
1      0.000    0.000    0.000    0.000 {built-in method builtins.print}
1      0.000    0.000    0.000    0.000 codecs.py:319(decode)
1000    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
1      0.000    0.000    0.000    0.000 {built-in method _codecs.utf_8_decode}
1      0.000    0.000    0.000    0.000 codecs.py:309(__init__)
1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1      0.000    0.000    0.000    0.000 codecs.py:260(__init__)
1      0.000    0.000    0.000    0.000 {built-in method builtins.len}

```

将存储容器由列表改为集合，利用集合哈希查找降低成员判断时间复杂度，完成程序优化。

```

words = open("words.txt", encoding="utf-8").read().split()
unique = set()
for word in words:
    unique.add(word)
print("count=", len(unique))

```

使用相同输入再运行两次，记录运行时间分别为 0.535s 和 0.520s

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ vi wordfreq.py

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m0.535s
user    0m0.015s
sys     0m0.031s

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ time python wordfreq.py
bash: syntax error near unexpected token `('
bash: $: command not found

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m0.520s
user    0m0.015s
sys     0m0.015s
```

(3) 实验结果

计算得优化前运行时间中位数为 1.637s，优化后运行时间中位数为 0.5275s，加速比为 3.1。

二、 课后练习

1. 题目一

(1) 题目内容

如果您希望某个进程结束后再开始另外一个进程，应该如何实现呢？在这个练习中，我们使用 `sleep 60 &` 作为先执行的程序。一种方法是使用 `wait` 命令。尝试启动这个休眠命令，然后待其结束后再执行 `ls` 命令。

(2) 解题过程

使用 `sleep 60 &` 将休眠程序放到后台运行；通过 `jobs` 确认后台任务处于运行状态；执行 `wait $!` 等待该后台进程结束，终端提示 `sleep` 任务完成后执行 `ls`。

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/1 (main)
$ sleep 60 &
[1] 378

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/1 (main)
$ jobs
[1]+  Running                  sleep 60 &

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/1 (main)
$ wait $!
[1]+  Done                      sleep 60

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/1 (main)
$ ls
```

(3) 实验结果

wait \$! 阻塞终端，直到 sleep 进程结束后才继续往下执行，符合实验预期。

2. 题目二

(1) 题目内容

创建一个 alias dc，让你把 cd 打错时也能正常工作。

(2) 解题过程

使用 alias 给 cd 取别名 dc，建立 test 文件夹并测试。

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/2 (main)
$ alias dc='cd'

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/2 (main)
$ mkdir test

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/2 (main)
$ dc test

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/2/test (main)
$
```

(3) 实验结果

测试通过，dc 具有和 cd 相同的功能。

3. 题目三

(1) 题目内容

完成一道 VimGolf 挑战：将 apple banana 两个单词的首字母改为大写，其余字母不变。

(2) 解题过程

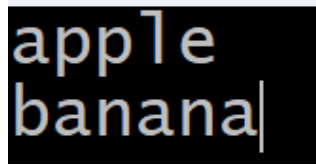
创建初始文本文件，使用 vim 打开；在普通模式使用 gUlwgU 按键组合完成两处首字母大写转换。

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/2 (main)
$ alias dc='cd'

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/2 (main)
$ mkdir test

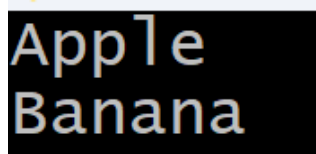
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/2 (main)
$ dc test

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/2/test (main)
$
```



(3) 实验结果

成功完成挑战。



4. 题目四

(1) 题目内容

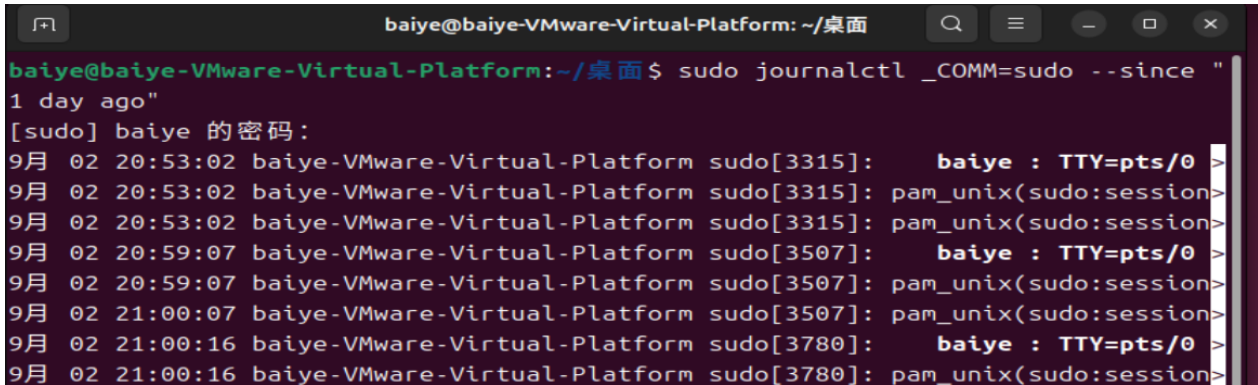
使用 Linux 上的 `journalctl` 或 macOS 上的 `log show` 命令来获取最近一天中超级用户的登录信息及其所执行的指令。如果找不到相关信息，您可以执行一些无害的命令，例如 `sudo ls` 然后再次查看。

(2) 解题过程

使用 Linux 上的 `journalctl` 命令获取最近一天中超级用户的登录信息。

```
baiye@baiye-VMware-Virtual-Platform: ~/桌面
baiye@baiye-VMware-Virtual-Platform:~/桌面$ sudo journalctl -u systemd-logind --
since "1 day ago"
[sudo] baiye 的密码:
9月 02 19:13:06 baiye-VMware-Virtual-Platform systemd[1]: Starting systemd-logi>
9月 02 19:13:06 baiye-VMware-Virtual-Platform systemd-logind[1041]: New seat se>
9月 02 19:13:06 baiye-VMware-Virtual-Platform systemd-logind[1041]: Watching sy>
9月 02 19:13:06 baiye-VMware-Virtual-Platform systemd-logind[1041]: Watching sy>
9月 02 19:13:06 baiye-VMware-Virtual-Platform systemd[1]: Started systemd-login>
9月 02 19:13:09 baiye-VMware-Virtual-Platform systemd-logind[1041]: New session>
9月 02 19:13:21 baiye-VMware-Virtual-Platform systemd-logind[1041]: New session>
9月 02 19:13:28 baiye-VMware-Virtual-Platform systemd-logind[1041]: Session c1 >
9月 02 19:13:28 baiye-VMware-Virtual-Platform systemd-logind[1041]: Removed ses>
-- Boot 24fc8eb021bc444e8d6a38a68b47f2bb --
9月 02 20:51:41 baiye-VMware-Virtual-Platform systemd[1]: Starting systemd-logi>
9月 02 20:51:42 baiye-VMware-Virtual-Platform systemd-logind[1089]: New seat se>
9月 02 20:51:42 baiye-VMware-Virtual-Platform systemd-logind[1089]: Watching sy>
9月 02 20:51:42 baiye-VMware-Virtual-Platform systemd-logind[1089]: Watching sy>
9月 02 20:51:42 baiye-VMware-Virtual-Platform systemd[1]: Started systemd-login>
9月 02 20:51:45 baiye-VMware-Virtual-Platform systemd-logind[1089]: New session>
9月 02 20:52:44 baiye-VMware-Virtual-Platform systemd-logind[1089]: New session>
9月 02 20:52:50 baiye-VMware-Virtual-Platform systemd-logind[1089]: Session c1 >
9月 02 20:52:50 baiye-VMware-Virtual-Platform systemd-logind[1089]: Removed ses>
lines 1-19/19 (END)
```

使用 Linux 上的 `journalctl` 命令获取最近一天中超级用户所执行的指令。



```

baiye@baiye-VMware-Virtual-Platform: ~/桌面
baiye@baiye-VMware-Virtual-Platform:~/桌面$ sudo journalctl _COMM=sudo --since "
1 day ago"
[sudo] baiye 的密码:
9月 02 20:53:02 baiye-VMware-Virtual-Platform sudo[3315]:    baiye : TTY=pts/0 >
9月 02 20:53:02 baiye-VMware-Virtual-Platform sudo[3315]: pam_unix(sudo:session>
9月 02 20:53:02 baiye-VMware-Virtual-Platform sudo[3315]: pam_unix(sudo:session>
9月 02 20:59:07 baiye-VMware-Virtual-Platform sudo[3507]:    baiye : TTY=pts/0 >
9月 02 20:59:07 baiye-VMware-Virtual-Platform sudo[3507]: pam_unix(sudo:session>
9月 02 21:00:07 baiye-VMware-Virtual-Platform sudo[3507]: pam_unix(sudo:session>
9月 02 21:00:16 baiye-VMware-Virtual-Platform sudo[3780]:    baiye : TTY=pts/0 >
9月 02 21:00:16 baiye-VMware-Virtual-Platform sudo[3780]: pam_unix(sudo:session>

```

(3) 实验结果

成功通过 `journalctl` 命令获取相关信息，如上图。

5. 题目五

(1) 题目内容

3. 安装 [shellcheck](#) 并尝试对下面的脚本进行检查。这段代码有什么问题吗？请修复相关问题。在您的编辑器中安装一个 linter 插件，这样它就可以自动地显示相关警告信息。

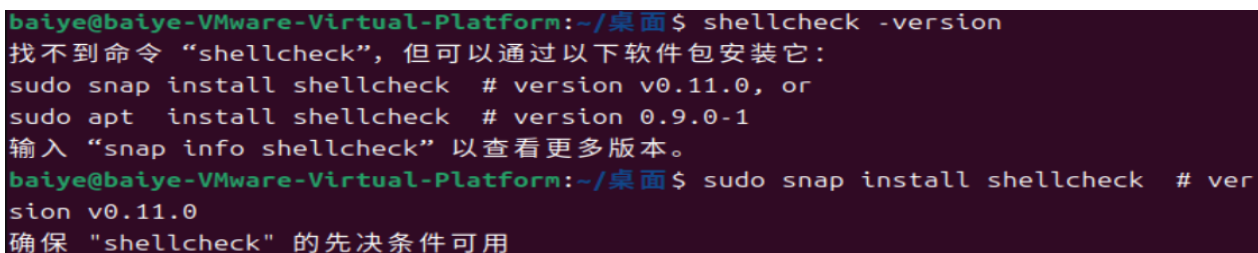
```

#!/bin/sh
## Example: a typical script with several problems
for f in $(ls *.m3u)
do
    grep -qi hq.*mp3 $f \
        && echo -e 'Playlist $f contains a HQ file in mp3 format'
done

```

(2) 解题过程

安装 `shellcheck`。



```

baiye@baiye-VMware-Virtual-Platform:~/桌面$ shellcheck -version
找不到命令 "shellcheck", 但可以通过以下软件包安装它:
sudo snap install shellcheck # version v0.11.0, or
sudo apt install shellcheck # version 0.9.0-1
输入 "snap info shellcheck" 以查看更多版本。
baiye@baiye-VMware-Virtual-Platform:~/桌面$ sudo snap install shellcheck # ver
sion v0.11.0
确保 "shellcheck" 的先决条件可用

```

使用 shellcheck 对脚本进行检查，得到问题如下。

```
baiye@baiye-VMware-Virtual-Platform:~/桌面$ shellcheck --version
ShellCheck - shell script analysis tool
version: 0.11.0
license: GNU General Public License, version 3
website: https://www.shellcheck.net
baiye@baiye-VMware-Virtual-Platform:~/桌面$ vi bad_script.sh
baiye@baiye-VMware-Virtual-Platform:~/桌面$ shellcheck bad_script.sh

In bad_script.sh line 3:
for f in $(ls *.m3u)
^-----^ SC2045 (error): Iterating over ls output is fragile. Use globs.
^-- SC2035 (info): Use ./*glob* or -- *glob* so names with dashes won't become options.

In bad_script.sh line 5:
grep -qi hq.*mp3 $f \
^-----^ SC2062 (warning): Quote the grep pattern so the shell won't interpret it.
^-- SC2086 (info): Double quote to prevent globbing and word splitting.

Did you mean:
grep -qi hq.*mp3 "$f" \

In bad_script.sh line 6:
&& echo -e 'Playlist $f contains a HQ file in mp3 format'
^-- SC3037 (warning): In POSIX sh, echo flags are undefined.
^-- SC2016 (info): Expressions don't expand in single quotes, use double quotes for that.
```

根据提示修复相关问题，修复后如下。

```
#!/bin/sh
## Example: fixed script
for f in *.m3u
do
    grep -qi "hq.*mp3" "$f" \
        && echo "Playlist $f contains a HQ file in mp3 format"
done
```

(3) 实验结果

再次使用 shellcheck 检查，没有告警弹出，实验完成。

```
baiye@baiye-VMware-Virtual-Platform:~/桌面$ vi bad_script.sh
baiye@baiye-VMware-Virtual-Platform:~/桌面$ shellcheck bad_script.sh
baiye@baiye-VMware-Virtual-Platform:~/桌面$
```

6. 题目六

(1) 题目内容

2. 这里有一些用于计算斐波那契数列 Python 代码，它为计算每个数字都定义了一个函数：

```
#!/usr/bin/env python
def fib0(): return 0

def fib1(): return 1

s = """def fib{}(): return fib{}() + fib{}()"""

if __name__ == '__main__':

    for n in range(2, 10):
        exec(s.format(n, n-1, n-2))
    # from functools import lru_cache
    # for n in range(10):
    #     exec("fib{} = lru_cache(1)(fib{})".format(n, n))
    print(eval("fib9()"))
```

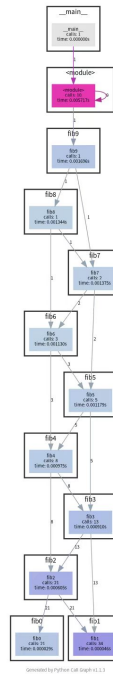
将代码拷贝到文件中使其变为一个可执行的程序。首先安装

[pycallgraph](#) 和 [graphviz](#) (如果您能够执行 dot, 则说明已经安装了 GraphViz.)。并使用 `pycallgraph graphviz -- ./fib.py` 来执行代码并查看 `pycallgraph.png` 这个文件。fib0 被调用了多少次？我们可以通过记忆法来对其进行优化。将注释掉的部分放开，然后重新生成图片。这回每个 fibN 函数被调用了多少次？

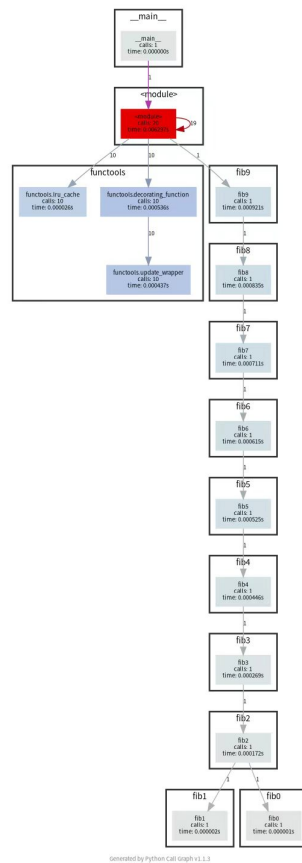
(2) 解题过程

安装 pycallgraph 和 graphviz 后，使用给定命令执行代码得到函数调用图。

```
baiye@baiye-VMware-Virtual-Platform:~/桌面$ vi bad_script.sh
baiye@baiye-VMware-Virtual-Platform:~/桌面$ shellcheck bad_script.sh
baiye@baiye-VMware-Virtual-Platform:~/桌面$
```

将注释部分放开，再次用相同命令执行代码得到函数调用图。



(3) 实验结果

优化前 fib0 被调用 21 次，优化后每个 fibN 函数仅调用 1 次，实验证明记忆法能够优化斐波那契数列的计算。

7. 题目七

(1) 题目内容

你可能见过像 `cmd -flag --notafalg` 这样的命令。这里的 `-` 是一个特殊参数，它告诉程序后面不要再继续解析选项 (flag) 了。也就是说，`-` 后面的所有内容都会被当作位置参数 (positional argument)。这有什么用？试着运行 `touch -myfile`，然后在不使用 `-` 的情况下把它删掉。

(2) 解题过程

执行 `touch -myfile` 创建名称以短横线开头的文件，直接执行 `rm -myfile` 发现文件名被识别为命令参数并报错。

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/7 (main)
$ touch -- -myfile

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/7 (main)
$ ls
-m myfile

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/7 (main)
$ rm -myfile
rm: unknown option -- m
Try 'rm ./-myfile' to remove the file '-myfile'.
Try 'rm --help' for more information.
```

使用相对路径 `rm ./-myfile`，不添加 `-`，成功删除文件。

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/7 (main)
$ rm ./-myfile

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/7 (main)
$ ls
```

(3) 实验结果

成功在不使用 `-` 的情况下删掉 `-myfile`。`-` 能终止命令行选项解析，把后续内容当作位置参数，从而可处理以 `-` 开头的文件名，避免被误识别为命令选项。

8. 题目八

(1) 题目内容

限制进程资源也是一个非常有用的技术。执行 `stress -c 3` 并使用 `htop` 对 CPU 消耗进行可视化。现在，执行 `taskset -cpu-list 0,2 stress -c 3` 并可视化。`stress` 占用了 3 个 CPU 吗？为什么没有？阅读 `man taskset` 来寻找答案。

(2) 解题过程

执行 `stress -c 3`，使用 `htop` 观测，3 个 CPU 核心全部满载。

```

baiye@baiye-VMware-Virtual-Platform: ~/桌面
baiye@baiye-VMware-Virtual-Platform:~/桌面$ stress -c 3
stress: info: [3874] dispatching hogs: 3 cpu, 0 io, 0 vm, 0 hdd

0[||||||||||||||||||||||||||||||||100.0%] Tasks: 127, 388 thr, 275 kthr; 3 runni
1[||||||||||||||||||||||||||||||||100.0%] Load average: 3.77 2.07 0.84
2[||||||||||||||||||||||||||||||||100.0%] Uptime: 00:02:54
Mem[||||||||||||||||||||||||||||1.04G/3.77G]
Swp[||||||||||||||||||||||||||||0K/3.59G]

Main I/O
  PID USER      PRI  NI  VIRT   RES   SHR  S  CPU% MEM%   TIME+  Command
  3875 baiye      20   0  3628    572   460  R  102.0  0.0    1:25.00 stress -c 3
  3876 baiye      20   0  3628    572   460  R   97.2  0.0    1:24.73 stress -c 3
  3877 baiye      20   0  3628    572   460  R   96.0  0.0    1:24.29 stress -c 3
  3938 baiye      20   0 11644   5564  3952  R    6.6  0.1    0:05.63 htop
  3650 root         20   0 1952M  51044 27036  S    1.2  1.3    0:02.40 /snap/snapd/c
  3891 root         20   0 1952M  51044 27036  S    1.2  1.3    0:01.32 /snap/snapd/c

```

执行 `taskset -cpu-list 0,2 stress -c 3`，`htop` 观察可见仅有 CPU0 与 CPU2 满载，CPU1 保持空闲。

```

baiye@baiye-VMware-Virtual-Platform: ~/桌面
baiye@baiye-VMware-Virtual-Platform:~/桌面$ stress -c 3
stress: info: [3874] dispatching hogs: 3 cpu, 0 io, 0 vm, 0 hdd
^C
baiye@baiye-VMware-Virtual-Platform:~/桌面$ taskset --cpu-list 0,2 stress -c 3
stress: info: [4252] dispatching hogs: 3 cpu, 0 io, 0 vm, 0 hdd

0[||||||||||||||||||||||||||||||||100.0%] Tasks: 126, 382 thr, 275 kthr; 3 runni
1[||||| 15.3%] Load average: 2.97 2.31 1.09
2[||||||||||||||||||||||||||||||||100.0%] Uptime: 00:05:04
Mem[||||||||||||||||||||||||||||1.05G/3.77G]
Swp[||||||||||||||||||||||||||||0K/3.59G]

Main I/O
  PID USER      PRI  NI  VIRT   RES   SHR  S  CPU% MEM%   TIME+  Command
  4255 baiye      20   0  3628    568   460  R   91.0  0.0    0:47.68 stress -c 3
  4254 baiye      20   0  3628    568   460  R   61.6  0.0    0:45.77 stress -c 3
  4253 baiye      20   0  3628    568   460  R   54.4  0.0    0:49.53 stress -c 3

```

阅读 `man taskset` 来寻找 `stress` 没有占用 3 个 CPU 的答案。

(3) 实验结果

`stress` 没有占用 3 个 CPU 的原因：`taskset -cpu-list 0,2` 限定该进程及其派生的所有子进程，只允许在编号 0、2 的 CPU 核心上调度运行。`stress -c 3` 创建 3 个 CPU 密集型进程，但是这 3 个进程被约束，不能调度到 CPU1。3 个进程只能在 CPU0 与 CPU2 两个核心上争抢时间片，所以最多只能跑满 2 个 CPU，无法占用第 3 个。

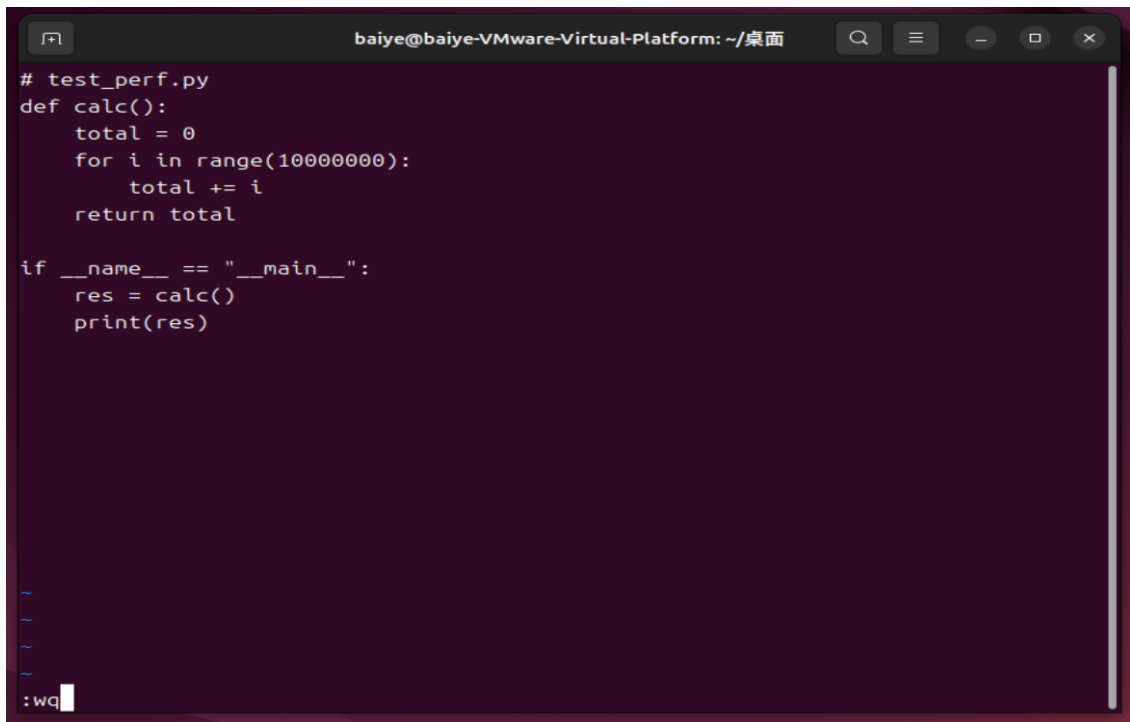
9. 题目九

(1) 题目内容

使用 perf stat 获取你所选程序的基础性能统计数据。这些不同的计数器分别代表什么含义？

(2) 解题过程

编写测试程序。

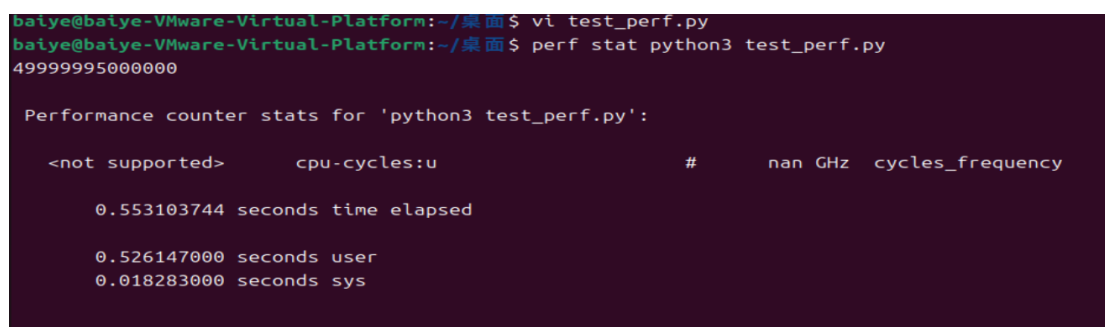


```
baiye@baiye-VMware-Virtual-Platform: ~/桌面
# test_perf.py
def calc():
    total = 0
    for i in range(100000000):
        total += i
    return total

if __name__ == "__main__":
    res = calc()
    print(res)

~
~
~
~
:wq
```

使用 perf stat 获取程序的基础性能统计数据。



```
baiye@baiye-VMware-Virtual-Platform:~/桌面$ vi test_perf.py
baiye@baiye-VMware-Virtual-Platform:~/桌面$ perf stat python3 test_perf.py
49999995000000

Performance counter stats for 'python3 test_perf.py':

   <not supported>      cpu-cycles:u          #          nan GHz  cycles_frequency

   0.553103744 seconds time elapsed

   0.526147000 seconds user
   0.018283000 seconds sys
```

(3) 实验结果

成功获得程序的基础性能统计数据如上图。各计数器含义：

time elapsed：程序从启动到结束总共消耗的墙钟时间。

user：用户态 CPU 时间，程序本身代码占用 CPU 时长。

sys：内核态 CPU 时间，操作系统内核为该程序执行的开销。

cpu-cycles:u：用户态 CPU 周期。

10. 题目十

(1) 题目内容

.假设你有一个很少失败的命令。为了调试它，你需要把它的输出保存下来，但等到一次失败运行可能会很耗时。写一个 bash 脚本，不断运行下面这个脚本直到它失败为止，并把标准输出和标准错误分别保存到文件里，最后把结果打印出来。如果你还能顺便报告它运行了多少次才失败，就加分。

```
#!/usr/bin/env bash

n=$(( RANDOM % 100 ))

if [[ n -eq 42 ]]; then
    echo "Something went wrong"
    >&2 echo "The error was using magic numbers"
    exit 1
fi

echo "Everything went according to plan"
```

(2) 解题过程

拷贝目标脚本并测试。

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/10 (main)
$ vi target.sh

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/10 (main)
$ chmod +x target.sh

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/10 (main)
$ ./target.sh
Everything went according to plan
```

编写循环脚本 `run_until_fail.sh`，设置计数器，循环调用目标脚本，将标准输出保存至 `out.txt`，标准错误保存至 `err.txt`。每次执行后检测退出状态，一旦脚本运行失败，输出总共执行次数，打印两个文件内的输出内容并终止循环。

```
#!/usr/bin/env bash
count=0
while true; do
    count=$((count + 1))
    ./target.sh > out.txt 2> err.txt
    if [ $? -ne 0 ]; then
        echo "脚本运行失败！一共执行了 $count 次"
        echo "====标准输出 out.txt===="
        cat out.txt
        echo "====标准错误 err.txt===="
        cat err.txt
        break
    fi
done
```

(3) 实验结果

经测试，脚本成功完成功能。

```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/10 (main)
$ vi run_until_fail.sh

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/10 (main)
$ chmod +x run_until_fail.sh

guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习/10 (main)
$ ./run_until_fail.sh
脚本运行失败！一共执行了 26 次
====标准输出 out.txt====
Something went wrong
====标准错误 err.txt====
The error was using magic numbers
```

11. 题目十一

(1) 题目内容

使用 AddressSanitizer 调试内存错误，将代码保存为 uaf.c。

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

int main() {
    char *greeting = malloc(32);
    strcpy(greeting, "Hello, world!");
    printf("%s\n", greeting);

    free(greeting);

    greeting[0] = 'J';
    printf("%s\n", greeting);

    return 0;
}
```

先不开启 sanitizer 编译并运行：程序看起来好像可以正常工作。然后使用 AddressSanitizer 编译，阅读报错信息。ASan 找到了什么 bug？修复这个识别出的问题。

(2) 解题过程

拷贝 uaf.c, 不开启 sanitizer 编译并运行, 程序正常工作。

```
baiye@baiye-VMware-Virtual-Platform:~/桌面$ vi uaf.c
baiye@baiye-VMware-Virtual-Platform:~/桌面$ gcc uaf.c -o uaf && ./uaf
Hello, world!
JB
```

然后使用 AddressSanitizer 编译, Asan 检测到堆内存释放后使用的 bug。

```
baiye@baiye-VMware-Virtual-Platform:~/桌面$ gcc -fsanitize=address -g uaf.c -o uaf && ./uaf
Hello, world!
=====
==7407==ERROR: AddressSanitizer: heap-use-after-free on address 0x503000000040 at pc 0x5e8c865d02ca bp 0x7fff8fd85e20 sp
0x7fff8fd85e10
WRITE of size 1 at 0x503000000040 thread T0
#0 0x5e8c865d02c9 in main /home/baiye/桌面/uaf.c:12
#1 0x71569862a1c9 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
#2 0x71569862a28a in __libc_start_main_impl ../csu/libc-start.c:360
#3 0x5e8c865d0184 in _start (/home/baiye/桌面/uaf+0x1184) (BuildId: 6f945eb900aeebb57086430ed7d131f44d8e1aa6)
0x503000000040 is located 0 bytes inside of 32-byte region [0x503000000040,0x503000000060)
freed by thread T0 here:
```

修复问题, 即删除 free 语句之后访问 greeting 指针的赋值与打印代码。

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

int main() {
    char *greeting = malloc(32);
    strcpy(greeting, "Hello, world!");
    printf("%s\n", greeting);

    free(greeting);

    return 0;
}
```

(3) 实验结果

再次使用 Asan 编译无报错, 修复成功。

三、 解题感悟

本次实验覆盖命令行环境、开发环境与工具、Debugging and Profiling 多个模块，进程信号控制、代码重构调试、性能分析、脚本检测等多项练习。在命令行实操中，我熟悉了后台任务管理、进程调度、特殊文件名处理等系统操作，理解了进程与信号的工作机制。在开发环境与工具层面，我实验了 VS Code 语义重构、ruff、shellcheck 静态检查。工具可以提前发现代码隐患，减少后期调试成本。调试与性能分析部分，我借助调试器定位归并排序逻辑缺陷，利用 cProfile 分析程序完成代码优化，使用 AddressSanitizer 捕获 C 语言释放后使用的内存漏洞。我意识到，代码能运行不等于代码正确，很多逻辑、内存、性能问题肉眼很难发现。善用各类调试、检测、性能分析工具，是工程开发里排查问题、提升代码质量的重要手段，也帮助我建立起更完整的开发思维。

四、 GitHub 链接和 commit 记录截图

1. GitHub 链接

<https://github.com/Youmoguolicheng/system-dev-tool>

2. commit 记录截图



```
guolicheng@LAPTOP-7D86CCBH MINGW64 ~/Desktop/系统开发工具基础/实验/week2/课后练习 (main)
$ git log --format="%ad %s" --date=iso
2026-09-04 22:51:43 +0800 完成课后第十一题
2026-09-03 15:17:14 +0800 完成课后第十题
2026-09-03 14:49:25 +0800 完成课后第九题
2026-09-03 14:16:47 +0800 完成课后第八题
2026-09-02 22:05:37 +0800 完成课后第六题
2026-09-02 21:23:43 +0800 完成课后第五题
2026-09-02 21:02:10 +0800 完成课后第四题
2026-09-02 20:40:55 +0800 完成课后第三题
2026-09-02 20:15:43 +0800 完成课后第一题
2026-09-02 11:40:16 +0800 完成部分实验报告
2026-08-31 16:23:21 +0800 完成第八题
2026-08-31 16:05:56 +0800 完成第七题
2026-08-31 14:59:14 +0800 完成第六题
2026-08-31 14:06:00 +0800 完成第五题
```